

Token Optimization — Summary for Management

StratRoom AI Agents · Date: 06 August 2026

1. Problem Being Solved

Every AI agent turn in StratRoom was unbounded on three fronts:

- **Input:** a user could paste an arbitrarily long message (up to 50,000 characters) — a single query could consume thousands of tokens.
- **Output:** each agent turn could make up to 4 LLM calls (1 initial + up to 3 tool-feedback loops), each allowed up to 2,048 tokens and 50,000 characters — no hard ceiling.
- **Tool results:** full database query results were fed back to the model verbatim, inflating every subsequent call in a conversation.

Result: unpredictable token spikes and cost per conversation.

2. What Was Implemented

A central token-governance layer was added at `backend/app/ai/tokens.py`, wired into every AI agent and LLM call site. All limits are configurable via environment variables (defaults shown):

MAX_CHAT_INPUT_CHARS	8,000	Rejects / truncates oversize inbound messages
MAX_RESPONSE_CHARS	16,000	Hard output cap applied to every response
MAX_LLM_MAX_TOKENS	2,048	Ceiling on provider max_tokens per call
MAX_TOOL_RESULT_CHARS	4,000	Caps tool results fed back to the model
TOKEN_BENCHMARK_WINDOW	200	Rolling window for token monitoring

How it works at runtime:

- **Input guard** — every chat message is sanitized (control characters removed, whitespace collapsed) and hard-truncated to 8,000 characters before it ever reaches the model. Oversized input is rejected with a clear message.
- **Output guard** — the unified LLM gateway (`llm_providers.py`) clamps max_tokens to 2,048 and truncates every response to 16,000 characters at the provider boundary, so all 12 agents are protected automatically. Truncated responses carry a visible “[Response truncated]” marker.
- **Tool-result cap** — database query results are trimmed to 4,000 characters before the model sees them on follow-up turns.
- **Benchmark logging** — every agent response records estimated input/output tokens to the `ai_agent_runs` table (previously always NULL), writes a structured “BENCH token_usage” log line, and updates an in-memory rolling benchmark. A new GET `/agents/benchmark` endpoint reports average tokens per response.

3. Measured Impact (Benchmark Report)

Estimated tokens use a characters/4 heuristic (monitoring-grade):

10 standard queries (avg)	~62	~4,800	~1,245	~1,245	0% (unchanged)
Spike: 45k-char paste	45,000	45,000	22,500	4,048	82.0%
Overall average	—	—	3,155	1,477	53.2%

Key message: Normal business queries are unaffected (0% change — no quality or UX impact). The caps only bite on abuse/spike scenarios — a 45,000-character paste now costs **82% fewer tokens** than before, and the worst-case token consumption per call is now bounded and predictable instead of open-ended.

4. Deliverables Delivered

- **Code:** tokens.py (new), config limits in config.py, enforcement in llm_providers.py, agents.py, ai.py and agents/base.py — covering all active AI agents.
- **Benchmark:** scripts/token_benchmark.py — reproducible before/after report (run: python scripts/token_benchmark.py).
- **Tests:** backend/tests/test_token_and_branding.py — 10/10 passing, covering input caps, control-character rejection, output truncation, max_tokens clamping and benchmark recording. Full suite shows no regressions (124/128; the 4 failures are pre-existing Docker / PostgreSQL-removal environment tests).
- **Monitoring:** GET /agents/benchmark endpoint plus per-response BENCH log lines for ongoing cost tracking.

Prepared from verified code and test results. All limits above are defaults and can be tuned via environment variables without code changes.